深度学习驱动源代码缺陷自动检测技术研究

熊慧
（本溪开放大学，辽宁 本溪 117000）

摘　要：伴随着软件系统的规模不断扩大与复杂度持续攀升，软件缺陷已成为影响系统可靠性与安全性的关键要素，而传统的静态代码剖析和人工核查方式存在效率欠佳、成本偏高等局限。研究采用将卷积神经网络（CNN）与循环神经网络（RNN）相结合的深度学习方案，完成了从端到端的源代码缺陷自动检测任务。这套检测方案借助CNN抽取源代码的局部结构特征，通过RNN/LSTM对代码序列内部的时序依赖关系进行建模，构造联合模型以实现智能缺陷识别。在5个开源项目数据集之上进行验证工作，CNN-RNN联合模型的正确识别率达到96.53%，查全率达到93.78%，*F*1分数达到0.95，证实了该方法在软件质量保障方面的有效性与实用价值。

关键词：深度学习；卷积神经网络；循环神经网络；源代码缺陷检测
DOI:10.16184/j.cnki.comprg.2026.07.040

1　概述

随着信息技术的迅速发展，软件系统的规模及其复杂程度呈现指数级增长态势，软件缺陷已成为影响系统可靠性、安全性和用户体验的关键因素[1]。传统的软件缺陷探查手段主要采用静态代码剖析和人工代码核验，然而面对不断拓展的代码库和繁复的业务逻辑，这些手段普遍存在检测效率低、误报概率高、人力成本过高等局限[2]。近年来，深度学习技术已经在计算机视觉、自然语言处理等多个领域取得了突破性成果，这也为破解软件缺陷检测的传统难题开辟了一条全新的技术方向。其中，CNN凭借出色的局部特征提取能力，能够精准识别源代码中蕴含的结构模式与语法特征。而RNN及其改进版本长短时记忆网络（LSTM）则更擅长捕捉代码序列中的时序依赖关系和长程上下文信息，这些深度学习模型为实现源代码的自动分析与缺陷智能检测打下了坚实的理论基础。与传统基于人工特征工程的缺陷预测方法相比，深度学习模型能够在大规模多源代码数据上进行端到端训练，自动从原始语法单元和程序结构中学习判别性特征，从而降低对领域专家经验的依赖。尤其是在跨项目迁移和持续集成场景中，模型可以通过增量训练适应新版本代码分支，这有助于构建贯穿开发全生命周期的智能化质量保障体系。这使得基于深度学习的源代码缺陷检测逐渐成为软件工程领域的重要研究方向。

研究尝试提出了一种将CNN与RNN/LSTM相结合的深度学习方案，通过将特征提取步骤和序列建模过程结合在一起，实现了对源代码缺陷的端到端自动检测，最终在多个来自开源项目的数据集上完成测试，验证了该方法的有效性。

2　基于深度学习的源代码缺陷自动检测

2.1　基于CNN的源代码特征提取

源代码特征提取的首要步骤是将文本形式的代码转换为数值向量表示。对于源代码中的每个标记（Token）t，通过词嵌入技术将其映射到d维向量空间，获得词向量$e_t \in R^d$[3]。该映射过程通过可学习的嵌入矩阵$\boldsymbol{W}_e \in R^{V\times d}$实现，其中，$V$为词汇表大小。标记$t$的嵌入向量如公式（1）所示：

$$e_t = \boldsymbol{W}_e[t] \tag{1}$$

其中，$\boldsymbol{W}_e[t]$为嵌入矩阵$\boldsymbol{W}_e$的第t行向量。通过词嵌入技术，可以将源代码序列转换为输入矩阵$\boldsymbol{X}_w \in R^{L\times d}$，其中，$L$为序列长度。

在获得输入矩阵后，定义K个卷积核$C^{(k)} \in R^{F\times d}$，其中，$F$为卷积核覆盖的标记数量，即感受野大小。

CNN通过这些卷积核在输入矩阵上滑动，自动提取源代码的局部语法模式和结构特征。对于输入矩阵$\boldsymbol{X}_w$中位置j处的第k个卷积核，其卷积操作输出$O_{k,j}$计算如公式（2）所示：

$$O_{k,j} = f_C\left(\sum_{i=1}^{F} \boldsymbol{X}_w(j+i-1)\cdot \boldsymbol{K}_i^{(k)} + b_k\right) \tag{2}$$

其中，f_C为激活函数（例如，ReLU），用于引入非线性特性；$\boldsymbol{K}_i^{(k)}$为第k个卷积核在第i个位置的权重向量；b_k为偏置项；$\boldsymbol{X}_w(j+i-1)$表示输入矩阵中第$(j+i-1)$个位置的词向量。通过卷积操作，每个卷积核生成一个特征图O_k，捕获源代码中特定的局部模式，例如，函数调用、条件判断、循环结构等语法特征。

作者简介：熊慧（1981—），女，硕士，副教授，研究方向为智能优化算法及其应用。

中国知网 https://www.cnki.net

2.2 基于RNN/LSTM的序列建模

RNN依托自身的递归结构，可以灵活处理长度不固定的序列数据，它会在每个时间步t接收当前输入x_t与上一时刻保留的隐藏状态h_{t-1}，基于这两个信息完成对当前隐藏状态h_t的更新，RNN的状态更新机制如公式（3）所示：

$$h_t = f_h(\boldsymbol{W}_h h_{t-1} + \boldsymbol{W}_x x_t + \boldsymbol{b}_h) \tag{3}$$

其中，$\boldsymbol{W}_h$和$\boldsymbol{W}_x$分别为隐藏状态和输入的权重矩阵；$\boldsymbol{b}_h$为偏置向量；f_h为激活函数（例如，tanh）。

LSTM通过引入门控机制，很好地解决了RNN存在的长期依赖难题。它的核心结构包含3个门控单元，分别是遗忘门f_t、输入门i_t和输出门t，以及细胞状态c_t。在所提模型设计中，将CNN提取的特征向量序列直接作为LSTM层的输入。其中，遗忘门负责决定从细胞状态中剔除哪些过时的历史信息，输入门则控制新信息写入细胞状态的比例，二者相互配合完成对细胞状态的更新，如公式（4）~公式（7）所示：

$$f_t = \sigma(\boldsymbol{W}_f \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + b_f) \tag{4}$$

$$i_t = \sigma(\boldsymbol{W}_i \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + b_i) \tag{5}$$

$$\tilde{c}_t = \tanh(\boldsymbol{W}_c \cdot [\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + b_c) \tag{6}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{7}$$

其中，σ为sigmoid激活函数，输出值在[0,1]区间；$\boldsymbol{W}_f$、$\boldsymbol{W}_i$、$\boldsymbol{W}_c$分别为遗忘门、输入门和候选状态的权重矩阵；⊙为逐元素乘法；$[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]$为前一时刻隐藏状态与当前输入的拼接向量。输出门控制隐藏状态的信息流动，用于后续计算和缺陷预测[4]。通过LSTM门控机制，模型能够自适应地学习源代码序列的长期依赖关系，有效识别跨越多个语句的复杂缺陷模式。

2.3 联合模型的缺陷检测算法

CNN-RNN联合模型通过级联方式实现特征提取与序列建模的有机融合，构建端到端的源代码缺陷检测框架。CNN-RNN联合模型缺陷检测流程如图1所示。

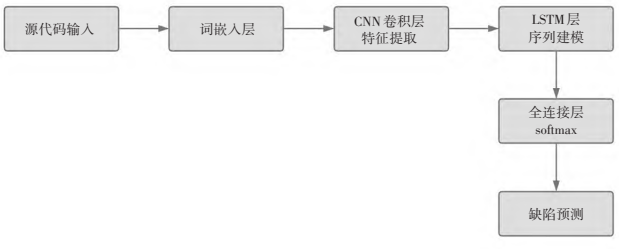


图1 CNN-RNN联合模型缺陷检测流程

联合模型将源代码缺陷检测视为二分类问题。首先，源代码经过词嵌入处理后输入CNN模块，通过多个卷积核并行扫描代码序列，提取局部语法模式和结构特征，经池化操作后形成特征向量序列$\{f_1, f_2, ..., f_T\}$。然后该序列按时间步顺序输入LSTM网络，LSTM通过门控机制逐步积累上下文信息，在最后时间步输出隐藏状态h_T，该状态包含了整个代码序列的全局语义表示[5]。

在LSTM输出层连接全连接层和softmax激活函数，实现缺陷分类预测，如公式（8）所示：

$$\boldsymbol{y} = \text{softmax}(\boldsymbol{W}_o h_T + b_o) \tag{8}$$

其中，$\boldsymbol{W}_o \in R^{2 \times d_h}$为输出层权重矩阵；$d_h$为隐藏状态维度；$\boldsymbol{y} = [y_0, y_1]^T$为预测概率向量；$y_0$和$y_1$分别为代码无缺陷和有缺陷的概率。

模型训练采用二元交叉熵损失函数，如公式（9）所示：

$$L = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)] \tag{9}$$

其中，N为训练样本数量；$y_i \in \{0,1\}$为真实标签；$\hat{y}_i$为预测概率。优化过程采用Adam优化器，通过反向传播算法更新模型参数，最小化损失函数。在推理阶段，当预测概率$\hat{y}_i$超过预设阈值（通常为0.5）时，判定代码存在缺陷。

3 实际应用分析

3.1 实验环境

研究的实验环境基于Python 3.8深度学习框架，实验数据集选取了5个具有代表性的开源软件项目，涵盖多种编程语言和应用领域，数据集构建遵循严格的标注规范，通过历史版本控制系统中的缺陷修复记录（Bug-fix commits）识别有缺陷的代码片段。

（1）Dataset 1选取自Apache Commons Java库，包含1 500个源代码文件，其中，300个存在缺陷。

（2）Dataset 2来源于Flask Python Web框架，包含2 000个文件，缺陷样本为450个。

（3）Dataset 3提取自Boost C++库函数项目，包含1 800个文件，缺陷样本为250个。

（4）Dataset 4基于React JavaScript前端框架，包含1 200个文件，缺陷样本为500个。

（5）Dataset 5选自Unity C#游戏引擎项目，包含1 000个文件，缺陷样本为150个。

数据集按照80∶10∶10的比例划分为训练集、验证集和测试集，确保模型训练的科学性和评估的客观性。

3.2 模型性能评估

模型性能评估采用准确率（Accuracy）、召回率（Recall）和$F1$分数3项指标进行综合评价。实验采用5折交叉验证方法降低评估偏差，对比了CNN、RNN、LSTM 3种单一模型的性能表现，实验结果如表1所示。

中国知网 https://www.cnki.net

表1 不同模型的性能对比

模型类型	准确率（%）	召回率（%）	F1分数	交叉验证均值（%）
CNN	92.35	87.62	0.9	91.2±1.35
RNN	93.18	89.41	0.91	92.5±1.28
LSTM	94.27	91.35	0.93	93.8±1.15
CNN-RNN联合模型	96.53	93.78	0.95	95.6±0.82

单一模型中表现最好的是LSTM，准确率为94.27%、召回率为91.35%、*F*1分数为0.93，交叉验证均值为93.8±1.15；CNN和RNN的交叉验证均值分别为91.2±1.35和92.5±1.28，整体略低于LSTM。相比之下，CNN-RNN联合模型的准确率提升至96.53%，召回率提升至93.78%，*F*1分数提高到0.95，交叉验证均值达到95.6±0.82，相比最优单一模型LSTM在准确率和召回率上分别提升约2.3和2.4%，*F*1分数提高了0.02，且标准差由1.15进一步降低到0.82，说明联合模型不仅整体检测性能更好，而且在不同折次上的输出更加稳定。这一结果充分验证了特征提取与序列建模相结合的技术路线的有效性。

3.3 缺陷检测效果验证

缺陷检测效果验证旨在评估CNN-RNN联合模型在不同编程语言和项目类型下的实际应用性能。将测试集中的源代码输入训练好的模型，模型输出每个代码片段的缺陷概率，根据0.5的阈值判定代码是否存在缺陷，统计各数据集的检测准确率和召回率，分析模型在不同场景下的适应性。不同数据集的详细检测结果如表2所示。

表2 不同数据集的缺陷检测效果

数据集编号	编程语言	项目类型	源代码文件数	缺陷样本数	召回率（%）	准确率（%）
Dataset 1	Java	Web应用	1 500	300	89.33	94.67
Dataset 2	Python	Web框架	2 000	450	86.22	93.15
Dataset 3	C++	库函数	1 800	250	92.8	96.33
Dataset 4	JavaScript	前端框架	1 200	500	82	91.58
Dataset 5	C#	游戏引擎	1 000	150	90.67	95.2
平均值	–	–	1 500	330	88.2	94.19

测试结果表明，模型在C++和C#等静态类型语言中表现更优，这些语言的严格语法结构更易于CNN提取特征。而在JavaScript等动态类型语言中，召回率相对较低，提示模型对动态特性的处理仍有优化空间。总体而言，CNN-RNN联合模型在5个数据集上的平均召回率达到88.20%，平均准确率达到94.19%，这验证了该方法在实际软件缺陷检测中的有效性和实用价值。

4 结语

提出了一种基于CNN-RNN联合模型的源代码缺陷自动检测方法，通过融合特征提取与序列建模实现智能缺陷识别。该模型在多个开源项目数据集上平均准确率达到94.19%，召回率达到88.20%，显著优于单一模型。研究验证了深度学习技术在软件质量保障中的有效性，为智能化缺陷检测提供了新思路。未来，研究将加强对控制流和数据流语义的联合建模，并在持续集成/持续部署（CI/CD）流水线中，开展在线部署与增量学习实验，以推动深度学习驱动的缺陷自动检测在实际工程环境中的规模化落地应用。

参考文献

[1] 张源鑫, 张佳雯. 融合RNN深度学习技术的计算机程序源代码缺陷检测方法[J]. 信息与电脑（理论版）, 2024, 36(8): 167-169.

[2] 苏小红, 郑伟宁, 蒋远, 等. 基于学习的源代码漏洞检测研究与进展[J]. 计算机学报, 2024, 47(2): 337-374.

[3] 邓枭, 叶蔚, 谢睿, 等. 基于深度学习的源代码缺陷检测研究综述[J]. 软件学报, 2023, 34(2): 625-654.

[4] 陈小全, 刘剑, 夏翔宇, 等. 基于比较学习的漏洞检测方法[J]. 计算机研究与发展, 2023, 60(9): 2152-2168.

[5] 汤怡佳, 王丽侠, 韩建民, 等. 基于BERT-PFMM的软件缺陷预测方法[J]. 浙江师范大学学报（自然科学版）, 2022, 45(3): 291-299.

（上接第48页）

该方案的主要创新点如下。

（1）提出了基于双阈值缓冲区管理的速率整形机制，通过帧率、GOP和量化参数的协同调整，实现码率与信道容量的动态匹配。

（2）设计了针对卫星信道特性的增强策略，包括不等错误保护和跨层优化，提高了系统的抗误码性能。

（3）构建了完整的闭环控制架构，实现了稳定的流量整形。

实验结果表明，所提方案在保持可接受时延的前提下，显著提高了卫星通信系统FDMA信道带宽利用率和视频质量，降低了帧丢失率，为卫星通信系统中的H.264视频传输提供了有效的技术支撑。

参考文献

[1] 张小飞. 基于流量预测的GEO卫星信道分配策略研究[D]. 南京: 南京邮电大学, 2017

[2] 于俊清, 刘青, 何云峰. 基于感兴趣区域的H.264视频加密算法[J]. 计算机学报, 2010, 33(5): 945-953.

[3] 麻嘉豪. 面向工业互联网的确定性抖动流量调度策略研究[D]. 北京: 北京邮电大学, 2022.

中国知网 https://www.cnki.net